



Terraform

Terraform AWS: Do Zero ao Avançado

Curso completo com 4 módulos

VPC • EC2 • RDS • S3 • IAM • State • Produção

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim

Terraform — Módulo 01

VPC Completa

Crie uma VPC completa na AWS com subnets públicas e privadas, Internet Gateway, NAT Gateway e Route Tables. Aprenda a estrutura de arquivos Terraform e os comandos essenciais.

Estrutura de Arquivos Terraform

Um projeto Terraform bem organizado separa responsabilidades em arquivos distintos. Essa convenção facilita a manutenção e colaboração em equipe.

File Tree **ESTRUTURA**

```
vpc-project/
├─ main.tf           # Recursos principais (VPC, subnets, gateways)
├─ variables.tf      # Declaração de variáveis de entrada
├─ outputs.tf        # Valores de saída após apply
├─ providers.tf      # Configuração do provider AWS
└─ terraform.tfvars  # Valores das variáveis (não commitar secrets)
```

Responsabilidades **CONVENÇÃO**

Arquivo	Função
main.tf	Recursos de infraestrutura (resource blocks)
variables.tf	Inputs do módulo com tipo, descrição e default
outputs.tf	Valores exportados para uso externo ou referência
providers.tf	Provider, versão do Terraform e backend
terraform.tfvars	Valores concretos das variáveis para o ambiente

Provider AWS

O bloco `terraform` define a versão mínima e os providers necessários. O bloco `provider` configura a região e credenciais.

providers.tf CONFIGURAÇÃO

```
terraform {  
  required_version = ">= 1.5.0"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = var.region  
  
  default_tags {  
    tags = {  
      Project      = var.project_name  
      ManagedBy    = "Terraform"  
      Environment  = "dev"  
    }  
  }  
}
```

Variables

Variáveis tornam o código reutilizável. Defina tipo, descrição e valores padrão para cada input.

variables.tf **INPUTS**

```
variable "project_name" {
  description = "Nome do projeto para tags e nomenclatura"
  type        = string
  default     = "cloudtrilhas"
}

variable "region" {
  description = "Região AWS para deploy dos recursos"
  type        = string
  default     = "us-east-1"
}

variable "vpc_cidr" {
  description = "CIDR block da VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnet_cidrs" {
  description = "Lista de CIDRs para subnets públicas"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "private_subnet_cidrs" {
  description = "Lista de CIDRs para subnets privadas"
  type        = list(string)
  default     = ["10.0.10.0/24", "10.0.20.0/24"]
}

variable "availability_zones" {
  description = "Lista de AZs para distribuir as subnets"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b"]
}
```

VPC Resources

O `main.tf` contém todos os recursos da VPC. Usamos `count` para criar múltiplas subnets de forma dinâmica.

VPC + Internet Gateway REDE BASE

```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.project_name}-vpc"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.project_name}-igw"
  }
}
```

Subnets Públicas COUNT

```
resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)

  vpc_id          = aws_vpc.main.id
  cidr_block      = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-${count.index + 1}"
  }
}
```

Subnets Privadas COUNT

```
resource "aws_subnet" "private" {
  count = length(var.private_subnet_cidrs)

  vpc_id          = aws_vpc.main.id
  cidr_block      = var.private_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]

  tags = {
    Name = "${var.project_name}-private-${count.index + 1}"
  }
}
```

NAT Gateway

SAÍDA PRIVADA

```
resource "aws_eip" "nat" {
  domain = "vpc"

  tags = {
    Name = "${var.project_name}-nat-eip"
  }
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id

  depends_on = [aws_internet_gateway.main]

  tags = {
    Name = "${var.project_name}-nat-gw"
  }
}
```

Route Tables – Pública

ROTEAMENTO

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "${var.project_name}-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  count = length(var.public_subnet_cidrs)

  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}
```


Route Tables – Privada

ROTEAMENTO

```
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = {
    Name = "${var.project_name}-private-rt"
  }
}

resource "aws_route_table_association" "private" {
  count = length(var.private_subnet_cidrs)

  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}
```

Outputs

Outputs exportam valores importantes após o `terraform apply`. Úteis para referência e integração com outros módulos.

outputs.tf SAÍDAS

```
output "vpc_id" {  
  description = "ID da VPC criada"  
  value       = aws_vpc.main.id  
}  
  
output "public_subnet_ids" {  
  description = "IDs das subnets públicas"  
  value       = aws_subnet.public[*].id  
}  
  
output "private_subnet_ids" {  
  description = "IDs das subnets privadas"  
  value       = aws_subnet.private[*].id  
}  
  
output "nat_gateway_ip" {  
  description = "IP público do NAT Gateway"  
  value       = aws_eip.nat.public_ip  
}
```

Comandos do Módulo

Workflow completo do Terraform — da inicialização à destruição dos recursos.

terraform init INICIALIZAR

Baixa providers, inicializa backend e prepara o diretório de trabalho.

```
terraform init
```

terraform fmt FORMATAR

Formata os arquivos .tf seguindo o estilo canônico do Terraform.

```
terraform fmt -recursive
```

terraform validate VALIDAR

Verifica a sintaxe e consistência da configuração sem acessar APIs.

```
terraform validate
```

terraform plan PLANEJAR

Mostra o que será criado, alterado ou destruído — sem executar nada.

```
terraform plan
```

terraform apply APLICAR

Executa as mudanças planejadas e cria/atualiza os recursos na AWS.

```
terraform apply

# Auto-approve (sem confirmação)
terraform apply -auto-approve
```

terraform output SAÍDAS

Exibe os valores definidos nos outputs após o apply.

```
terraform output

# Output específico
terraform output vpc_id
```

terraform state list ESTADO

Lista todos os recursos gerenciados no state file.

```
terraform state list
```

terraform state show DETALHE

Mostra atributos detalhados de um recurso específico no state.

```
terraform state show aws_vpc.main
```

terraform destroy DESTRUIR

Remove todos os recursos gerenciados pelo Terraform. Use com cuidado!

```
terraform destroy

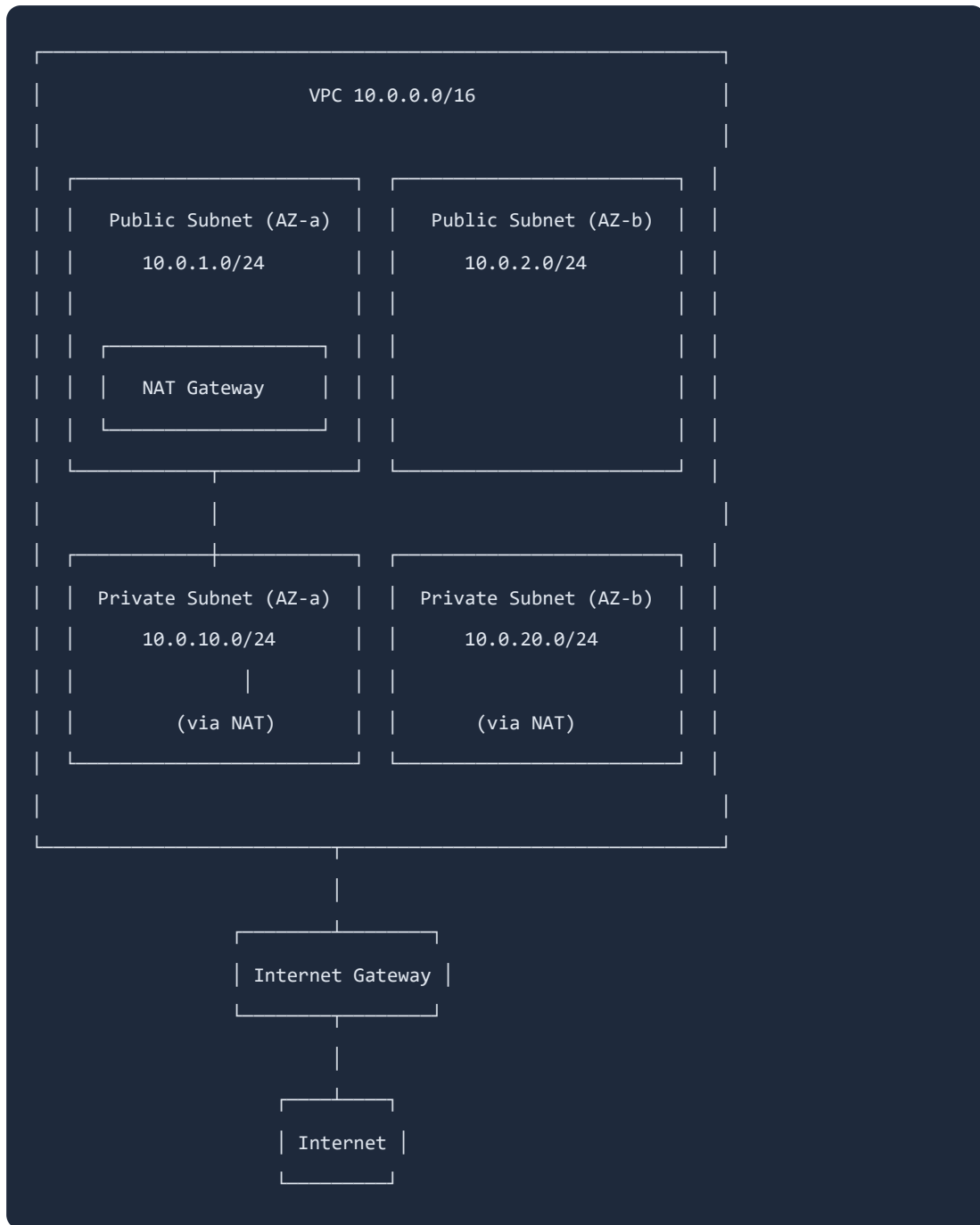
# Auto-approve
terraform destroy -auto-approve
```

Diagrama de Arquitetura

Visão geral da VPC completa criada neste módulo:

VPC Architecture

DIAGRAMA



 Terraform — Módulo 02

VPC + EC2

Adicione instâncias EC2 à sua VPC com Security Groups, Key Pairs e User Data. Aprenda Data Sources e funções Terraform.

Data Source AMI

Data Sources permitem buscar informações da AWS sem criar recursos. Aqui buscamos a AMI mais recente do Amazon Linux 2023 dinamicamente.

data "aws_ami"

DATA SOURCE

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }

  filter {
    name     = "virtualization-type"
    values   = ["hvm"]
  }

  filter {
    name     = "state"
    values   = ["available"]
  }
}
```

Referência **uso**

Use `data.aws_ami.amazon_linux.id` para referenciar o ID da AMI encontrada. Isso garante que sempre usamos a versão mais recente sem hardcoding.

```
# Acessar atributos do data source
data.aws_ami.amazon_linux.id
data.aws_ami.amazon_linux.name
data.aws_ami.amazon_linux.description
```

Security Group

Security Groups atuam como firewall virtual para suas instâncias EC2. Defina regras de ingress (entrada) e egress (saída).

aws_security_group FIREWALL


```

variable "my_ip" {
  description = "Seu IP público para acesso SSH (formato: x.x.x.x/32)"
  type        = string
}

resource "aws_security_group" "web" {
  name          = "${var.project_name}-web-sg"
  description   = "Security Group para instancia web"
  vpc_id        = aws_vpc.main.id

  # HTTP
  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # HTTPS
  ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # SSH - restrito ao seu IP
  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  # Egress - permitir todo tráfego de saída
  egress {
    description = "Allow all outbound"
    from_port   = 0
  }
}

```

```

    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.project_name}-web-sg"
  }
}

```

Key Pair

Key Pairs permitem acesso SSH às instâncias EC2. Use a função `file()` para ler a chave pública do disco.

`aws_key_pair` `ssh`

```

# Gerar key pair localmente (se ainda não tiver):
# ssh-keygen -t ed25519 -f ~/.ssh/terraform-key -C "terraform"

resource "aws_key_pair" "main" {
  key_name      = "${var.project_name}-key"
  public_key   = file("~/.ssh/terraform-key.pub")

  tags = {
    Name = "${var.project_name}-key"
  }
}

```

Função file() **BUILT-IN**

A função `file(path)` lê o conteúdo de um arquivo no disco local durante o plan/apply. Ideal para chaves públicas, scripts e templates.

```
# Exemplos de uso da função file()
file("~/ssh/id_rsa.pub")
file("${path.module}/scripts/init.sh")
file("${path.root}/configs/policy.json")
```

EC2 Instance

O recurso `aws_instance` cria uma instância EC2 na subnet pública com todas as configurações de rede e inicialização.

aws_instance **COMPUTE**

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.public[0].id
  key_name      = aws_key_pair.main.key_name
  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = file("${path.module}/scripts/user-data.sh")

  tags = {
    Name = "${var.project_name}-web-server"
  }
}
```

Alternativa: Heredoc INLINE

Para scripts curtos, você pode usar heredoc diretamente no `user_data`:

```
resource "aws_instance" "web" {  
  ami           = data.aws_ami.amazon_linux.id  
  instance_type = "t3.micro"  
  subnet_id     = aws_subnet.public[0].id  
  key_name      = aws_key_pair.main.key_name  
  vpc_security_group_ids = [aws_security_group.web.id]  
  
  user_data = <<-EOF  
    #!/bin/bash  
    yum update -y  
    yum install -y httpd  
    systemctl start httpd  
    systemctl enable httpd  
    echo "<h1>Hello from Terraform!</h1>" > /var/www/html/index.html  
  EOF  
  
  tags = {  
    Name = "${var.project_name}-web-server"  
  }  
}
```

User Data Script

O User Data é executado automaticamente na primeira inicialização da instância (como root). Ideal para instalar pacotes e configurar serviços.

scripts/user-data.sh **BOOTSTRAP**

```
#!/bin/bash
set -euxo pipefail

# Atualizar pacotes do sistema
yum update -y

# Instalar Apache HTTP Server
yum install -y httpd

# Iniciar e habilitar o serviço
systemctl start httpd
systemctl enable httpd

# Criar página inicial
cat <<'HTML' > /var/www/html/index.html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>CloudTrilhas - EC2 com Terraform</title>
  <style>
    body { font-family: sans-serif; text-align: center; padding: 50px; }
    h1 { color: #2563eb; }
  </style>
</head>
<body>
  <h1>🚀 EC2 provisionada com Terraform!</h1>
  <p>Hostname: $(hostname)</p>
  <p>IP Privado: $(hostname -I | awk '{print $1}')</p>
  <p>Data: $(date)</p>
</body>
</html>
HTML

echo "User data script completed successfully"
```

Outputs

Outputs da instância EC2 para acesso e referência rápida.

outputs.tf SAÍDAS

```
output "instance_id" {  
  description = "ID da instância EC2"  
  value       = aws_instance.web.id  
}  
  
output "public_ip" {  
  description = "IP público da instância"  
  value       = aws_instance.web.public_ip  
}  
  
output "public_dns" {  
  description = "DNS público da instância"  
  value       = aws_instance.web.public_dns  
}
```

Acessar a instância uso

```
# Após terraform apply, acesse via SSH:  
ssh -i ~/.ssh/terraform-key ec2-user@$(terraform output -raw public_ip)  
  
# Ou acesse o web server no navegador:  
# http://$(terraform output -raw public_dns)
```

Novos Comandos Aprendidos

Comandos avançados para controle granular do Terraform.

terraform plan -out **SALVAR PLANO**

Salva o plano de execução em um arquivo binário. Garante que o apply executará exatamente o que foi revisado.

```
# Salvar plano em arquivo
terraform plan -out=plan.tfplan

# Visualizar plano salvo
terraform show plan.tfplan
```

terraform apply plan.tfplan **APLICAR PLANO**

Aplica um plano salvo anteriormente. Não pede confirmação pois o plano já foi revisado.

```
# Aplicar plano salvo (sem prompt de confirmação)
terraform apply plan.tfplan
```

terraform apply -target **RECURSO ESPECÍFICO**

Aplica mudanças apenas em um recurso específico. Útil para debugging e deploys parciais.

```
# Aplicar apenas a instância EC2
terraform apply -target=aws_instance.web

# Aplicar apenas o security group
terraform apply -target=aws_security_group.web

# Múltiplos targets
terraform apply \
  -target=aws_security_group.web \
  -target=aws_instance.web
```

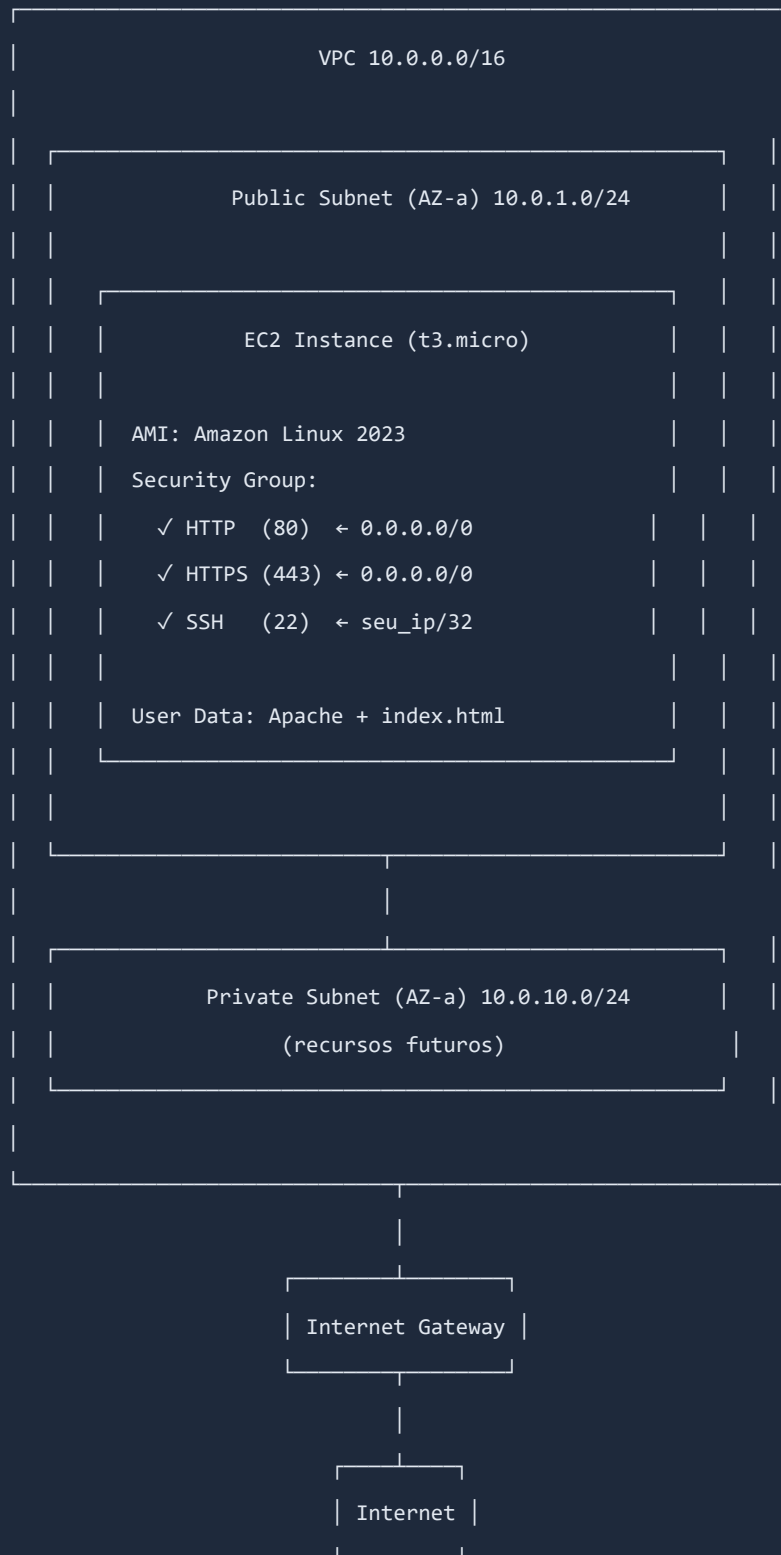
 **Atenção:** O uso de `-target` deve ser temporário. Em produção, sempre aplique o plano completo para manter o state consistente com a configuração.

Diagrama de Arquitetura

Visão geral da VPC com instância EC2 na subnet pública:

VPC + EC2 Architecture

DIAGRAMA



Usuário → Internet → IGW → Public Subnet → EC2 (porta 80/443)

Admin → Internet → IGW → Public Subnet → EC2 (porta 22/SSH)

 Terraform — Módulo 03

VPC + EC2 + RDS

Adicione um banco de dados RDS MySQL à infraestrutura. Aprenda DB Subnet Groups, Security Groups com referência cruzada, variáveis sensíveis e Locals.

Locals — Tags Reutilizáveis

O bloco `locals` permite definir valores reutilizáveis dentro do módulo. Ideal para tags comuns que serão aplicadas em todos os recursos.

`locals { }` [LOCALS.TF](#)

Defina um mapa de tags comuns para manter consistência em toda a infraestrutura.

```
locals {
  common_tags = {
    Project      = "cloudtrilhas"
    Environment = var.environment
    ManagedBy    = "terraform"
  }
}
```

Uso das Tags [REFERÊNCIA](#)

Aplique as tags em qualquer recurso usando `merge()` para combinar com tags específicas.

```
# Uso direto
tags = local.common_tags

# Merge com tags adicionais
tags = merge(local.common_tags, {
  Name = "meu-recurso"
  Role = "database"
})
```

DB Subnet Group

O RDS exige um **DB Subnet Group** que define em quais subnets o banco pode ser provisionado. Para segurança, usamos as subnets privadas.

`aws_db_subnet_group` RDS.TF

Agrupar as subnets privadas para que o RDS seja lançado em uma rede isolada sem acesso público.

```
resource "aws_db_subnet_group" "main" {
  name      = "${var.project_name}-db-subnet-group"
  subnet_ids = [
    aws_subnet.private_a.id,
    aws_subnet.private_b.id
  ]

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-db-subnet-group"
  })
}
```

RDS Security Group — Referência Cruzada

O Security Group do RDS permite tráfego **apenas** da instância EC2, usando `source_security_group_id` em vez de CIDR blocks. Isso garante que somente a EC2 autorizada acesse o banco.

aws_security_group **SG-RDS.TF**

Permite porta 3306 (MySQL) apenas do Security Group da EC2 — referência cruzada entre SGs.

```
resource "aws_security_group" "rds" {
  name           = "${var.project_name}-rds-sg"
  description    = "Security group for RDS MySQL"
  vpc_id        = aws_vpc.main.id

  ingress {
    description      = "MySQL from EC2"
    from_port        = 3306
    to_port          = 3306
    protocol          = "tcp"
    source_security_group_id = aws_security_group.ec2.id
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-rds-sg"
  })
}
```

Referência Cruzada CONCEITO

Em vez de abrir para um CIDR, referenciamos o SG da EC2 diretamente. Se o IP da EC2 mudar, a regra continua válida.

```
# ❌ Evite – IP pode mudar
cidr_blocks = ["10.0.1.50/32"]

# ✅ Correto – referência ao SG
source_security_group_id = aws_security_group.ec2.id
```

RDS Instance — MySQL 8.0

O recurso `aws_db_instance` provisiona o banco de dados gerenciado. Configuramos criptografia, backup e isolamento de rede.

aws_db_instance **RDS.TF**

Instância RDS MySQL com todas as boas práticas de segurança e isolamento.

```
resource "aws_db_instance" "main" {  
  identifier      = "${var.project_name}-mysql"  
  engine         = "mysql"  
  engine_version = "8.0"  
  instance_class = var.db_instance_class  
  
  allocated_storage = var.db_allocated_storage  
  storage_type      = "gp3"  
  
  db_name      = var.db_name  
  username     = var.db_username  
  password     = var.db_password  
  
  db_subnet_group_name = aws_db_subnet_group.main.name  
  vpc_security_group_ids = [aws_security_group.rds.id]  
  
  multi_az          = false  
  publicly_accessible = false  
  skip_final_snapshot = true  
  
  backup_retention_period = var.db_backup_retention  
  storage_encrypted      = true  
  
  tags = merge(local.common_tags, {  
    Name = "${var.project_name}-mysql"  
    Role = "database"  
  })  
}
```

Parâmetros Importantes REFERÊNCIA

Parâmetro	Descrição
<code>storage_type = "gp3"</code>	SSD de nova geração, melhor custo-benefício
<code>multi_az = false</code>	Lab: single AZ. Produção: use true
<code>publicly_accessible = false</code>	Sem IP público — acesso apenas via VPC
<code>skip_final_snapshot = true</code>	Lab: sem snapshot ao destruir. Produção: false
<code>storage_encrypted = true</code>	Criptografia at-rest com KMS padrão
<code>backup_retention_period</code>	Dias de retenção de backups automáticos

Variáveis Sensíveis

Senhas e segredos **nunca** devem ser commitados no código. O Terraform oferece o atributo `sensitive = true` para proteger valores no output.

variable "db_password" VARIABLES.TF

Declare a variável como sensível para que o Terraform oculte seu valor nos logs e outputs.

```
variable "db_password" {
  description = "Password for the RDS MySQL instance"
  type        = string
  sensitive   = true

  validation {
    condition     = length(var.db_password) >= 8
    error_message = "A senha deve ter no mínimo 8 caracteres."
  }
}
```

Formas de Passar o Valor SEGURANÇA

Existem várias formas seguras de fornecer variáveis sensíveis sem expor no código.

```
# Opção 1: Variável de ambiente (recomendado em CI/CD)
export TF_VAR_db_password="MinhaSenhaSegura123!"
terraform apply

# Opção 2: Arquivo .tfvars (NÃO commitar!)
# secrets.tfvars
db_password = "MinhaSenhaSegura123!"

terraform apply -var-file="secrets.tfvars"

# Opção 3: Prompt interativo
terraform apply
# var.db_password
#   Enter a value: *****

# Opção 4: AWS Secrets Manager (produção)
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "prod/db/password"
}
# Uso: data.aws_secretsmanager_secret_version.db.secret_string
```

 **Importante:** Adicione *.tfvars e .terraform/ ao seu .gitignore. Nunca committe arquivos com senhas no repositório.

Outputs — Endpoint e Porta

Outputs exportam valores úteis após o `terraform apply`. O endpoint do RDS é essencial para configurar a aplicação na EC2.

output **OUTPUTS.TF**

Exporte o endpoint e a porta do banco para uso na configuração da aplicação.

```
output "db_endpoint" {
  description = "RDS MySQL endpoint address"
  value      = aws_db_instance.main.endpoint
}

output "db_port" {
  description = "RDS MySQL port"
  value      = aws_db_instance.main.port
}

output "db_name" {
  description = "Database name"
  value      = aws_db_instance.main.db_name
}

# Exemplo de uso na EC2 (user_data):
# mysql -h ${output.db_endpoint} -P ${output.db_port} \
#   -u ${var.db_username} -p${var.db_password} ${var.db_name}
```

Diagrama de Arquitetura

Visão geral da comunicação entre EC2 (subnet pública) e RDS (subnet privada) dentro da VPC.

Arquitetura VPC + EC2 + RDS DIAGRAMA

 Terraform — Módulo 04

VPC + EC2 + RDS + S3

Complete a arquitetura com S3 Bucket, IAM Role para EC2, Bucket Policy, Lifecycle Rules. Aprenda terraform import e conceitos de produção.

S3 Bucket — Configuração Segura

O S3 armazena objetos (arquivos, backups, assets). Configuramos versionamento, criptografia e bloqueio de acesso público seguindo as melhores práticas.

aws_s3_bucket S3.TF

Cria o bucket base. No Terraform AWS Provider v4+, configurações são recursos separados.

```
resource "aws_s3_bucket" "app" {
  bucket = "${var.project_name}-${var.environment}-assets"

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-${var.environment}-assets"
    Role = "storage"
  })
}
```

aws_s3_bucket_versioning VERSIONAMENTO

Habilita versionamento para manter histórico de alterações e permitir recuperação.

```
resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

aws_s3_bucket_server_side_encryption_configuration CRIPTOGRAFIA

Criptografia at-rest com AES256 (SSE-S3). Todos os objetos são criptografados automaticamente.

```
resource "aws_s3_bucket_server_side_encryption_configuration" "app" {  
  bucket = aws_s3_bucket.app.id  
  
  rule {  
    apply_server_side_encryption_by_default {  
      sse_algorithm = "AES256"  
    }  
    bucket_key_enabled = true  
  }  
}
```

aws_s3_bucket_public_access_block BLOQUEIO PÚBLICO

Bloqueia todo acesso público ao bucket — todas as 4 opções habilitadas.

```
resource "aws_s3_bucket_public_access_block" "app" {  
  bucket = aws_s3_bucket.app.id  
  
  block_public_acls      = true  
  block_public_policy    = true  
  ignore_public_acls     = true  
  restrict_public_buckets = true  
}
```

S3 Lifecycle Rules — Otimização de Custos

Lifecycle Rules movem objetos automaticamente entre classes de armazenamento e expiram dados antigos, reduzindo custos significativamente.

aws_s3_bucket_lifecycle_configuration LIFECYCLE.TF

Transições automáticas: Standard → Standard-IA (30d) → Glacier (90d) → Expiração (365d).

```
resource "aws_s3_bucket_lifecycle_configuration" "app" {
  bucket = aws_s3_bucket.app.id

  rule {
    id      = "transition-and-expire"
    status = "Enabled"

    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }

    transition {
      days          = 90
      storage_class = "GLACIER"
    }

    expiration {
      days = 365
    }

    noncurrent_version_expiration {
      noncurrent_days = 30
    }

    abort_incomplete_multipart_upload {
      days_after_initiation = 7
    }
  }
}
```

Classes de Armazenamento REFERÊNCIA

Classe	Uso	Custo
STANDARD	Acesso frequente	\$\$\$\$
STANDARD_IA	Acesso infrequente	\$\$\$
GLACIER	Arquivo de longo prazo	\$\$
DEEP_ARCHIVE	Raramente acessado	\$

IAM Role para EC2 — Acesso ao S3

Em vez de usar Access Keys na EC2, criamos uma **IAM Role** com Instance Profile. A EC2 assume a role automaticamente via metadata service.

aws_iam_role IAM.TF

Role com trust policy permitindo que o serviço EC2 assuma esta role.

```
resource "aws_iam_role" "ec2_s3_role" {
  name = "${var.project_name}-ec2-s3-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com"
        }
      }
    ]
  })

  tags = local.common_tags
}
```

aws_iam_role_policy **PERMISSÕES S3**

Policy inline com permissões específicas para o bucket da aplicação.

```
resource "aws_iam_role_policy" "ec2_s3_policy" {
  name = "${var.project_name}-ec2-s3-policy"
  role = aws_iam_role.ec2_s3_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "s3:GetObject",
          "s3:PutObject",
          "s3:ListBucket",
          "s3:DeleteObject"
        ]
        Resource = [
          aws_s3_bucket.app.arn,
          "${aws_s3_bucket.app.arn}/*"
        ]
      }
    ]
  })
}
```

aws_iam_instance_profile INSTANCE PROFILE

O Instance Profile é o "wrapper" que conecta a IAM Role à instância EC2.

```
resource "aws_iam_instance_profile" "ec2_profile" {
  name = "${var.project_name}-ec2-profile"
  role = aws_iam_role.ec2_s3_role.name

  tags = local.common_tags
}

# Uso na EC2:
resource "aws_instance" "web" {
  # ... outras configurações ...

  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
}
```

Bucket Policy — Controle de Acesso

A Bucket Policy define regras de acesso no nível do bucket. Combinamos permissão para a EC2 Role com negação de transporte inseguro (HTTP).

`aws_s3_bucket_policy` **S3-POLICY.TF**

Permite acesso da EC2 Role e nega requisições sem HTTPS (transporte inseguro).

```

resource "aws_s3_bucket_policy" "app" {
  bucket = aws_s3_bucket.app.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AllowEC2RoleAccess"
        Effect   = "Allow"
        Principal = {
          AWS = aws_iam_role.ec2_s3_role.arn
        }
        Action = [
          "s3:GetObject",
          "s3:PutObject",
          "s3:ListBucket"
        ]
        Resource = [
          aws_s3_bucket.app.arn,
          "${aws_s3_bucket.app.arn}/*"
        ]
      },
      {
        Sid      = "DenyInsecureTransport"
        Effect   = "Deny"
        Principal = "*"
        Action    = "s3:*"
        Resource = [
          aws_s3_bucket.app.arn,
          "${aws_s3_bucket.app.arn}/*"
        ]
        Condition = {
          Bool = {
            "aws:SecureTransport" = "false"
          }
        }
      }
    ]
  })
}

```

```
  })  
}
```

Comandos Avançados do Terraform

Comandos essenciais para gerenciar infraestrutura em cenários reais: planejamento, importação e workspaces.

`terraform plan -out` PLANEJAMENTO

Salva o plano em arquivo binário para aplicar exatamente o que foi revisado.

```
# Gerar plano e salvar em arquivo  
terraform plan -out=plan.tfplan  
  
# Aplicar exatamente o plano salvo (sem re-calcular)  
terraform apply plan.tfplan
```

`terraform output -json` OUTPUTS

Exporta outputs em formato JSON para integração com scripts e CI/CD.

```
# Ver todos os outputs em JSON  
terraform output -json  
  
# Output específico (sem aspas)  
terraform output -raw db_endpoint  
  
# Usar em scripts bash  
DB_HOST=$(terraform output -raw db_endpoint)  
echo "Conectando em: $DB_HOST"
```

terraform import IMPORTAÇÃO

Importa recursos existentes (criados manualmente) para o state do Terraform.

```
# Sintaxe: terraform import RECURSO_TF ID_AWS
terraform import aws_s3_bucket.app meu-bucket-existente

terraform import aws_db_instance.main meu-rds-identifier

terraform import aws_vpc.main vpc-0abc123def456

# Após importar, escreva o código .tf correspondente
# e execute terraform plan para verificar drift
```

terraform state mv STATE

Move recursos no state — útil ao refatorar código sem destruir infraestrutura.

```
# Renomear recurso no state
terraform state mv aws_instance.web aws_instance.app_server

# Mover para módulo
terraform state mv aws_s3_bucket.app module.storage.aws_s3_bucket.main

# Listar recursos no state
terraform state list

# Ver detalhes de um recurso
terraform state show aws_s3_bucket.app
```

terraform workspace WORKSPACES

Workspaces permitem múltiplos states no mesmo código — útil para ambientes dev/staging/prod.

```
# Criar novo workspace
terraform workspace new dev
terraform workspace new staging
terraform workspace new prod

# Listar workspaces
terraform workspace list

# Selecionar workspace
terraform workspace select dev

# Usar no código
locals {
    environment = terraform.workspace
}

resource "aws_instance" "web" {
    instance_type = terraform.workspace == "prod" ? "t3.medium" : "t3.micro"
}
```

Lab vs Produção — Diferenças Críticas

O código de lab é simplificado para aprendizado. Em produção, diversas configurações adicionais são necessárias para segurança, disponibilidade e compliance.

Comparação Lab × Produção REFERÊNCIA

Aspecto	Lab	Produção
Multi-AZ (RDS)	false	true (alta disponibilidade)
Deletion Protection	Desabilitado	Habilitado em RDS e S3
Backup Retention	1-7 dias	14-35 dias
Instance Types	t3.micro / db.t3.micro	t3.medium+ / db.r6g.large+
State Backend	Local	S3 + DynamoDB (remoto)
Final Snapshot	skip_final_snapshot = true	skip_final_snapshot = false
Logs	Básico	CloudWatch + VPC Flow Logs
Secrets	TF_VAR ou .tfvars	AWS Secrets Manager / SSM
Monitoring	Opcional	CloudWatch Alarms + SNS
WAF	Não	AWS WAF no ALB/CloudFront

Remote Backend — State Compartilhado

Em equipe ou CI/CD, o state deve ser armazenado remotamente com locking para evitar conflitos. O backend S3 + DynamoDB é o padrão na AWS.

terraform backend "s3" BACKEND.TF

Configuração do backend remoto com criptografia e locking via DynamoDB.

```
terraform {
  backend "s3" {
    bucket      = "cloudtrilhas-terraform-state"
    key         = "vpc-ec2-rds-s3/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt     = true
  }
}
```

Pré-requisitos do Backend SETUP

O bucket S3 e a tabela DynamoDB devem existir ANTES de configurar o backend.

```
# Criar bucket para state (via AWS CLI ou console)
aws s3api create-bucket \
  --bucket cloudtrilhas-terraform-state \
  --region us-east-1

# Habilitar versionamento
aws s3api put-bucket-versioning \
  --bucket cloudtrilhas-terraform-state \
  --versioning-configuration Status=Enabled

# Criar tabela DynamoDB para locking
aws dynamodb create-table \
  --table-name terraform-state-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST

# Após criar, inicialize o backend
terraform init -migrate-state
```

 **Dica:** O DynamoDB table usa `LockID` como partition key. Quando alguém executa `terraform apply`, um lock é criado impedindo execuções simultâneas.

Estrutura Completa do Projeto

Organização recomendada dos arquivos `.tf` para o projeto VPC + EC2 + RDS + S3.

Árvore de Arquivos

ESTRUTURA

```
terraform-vpc-ec2-rds-s3/
├─ main.tf           # Provider e configurações gerais
├─ backend.tf        # Backend remoto S3 + DynamoDB
├─ variables.tf      # Declaração de todas as variáveis
├─ locals.tf         # Locals (common_tags, etc.)
├─ outputs.tf        # Outputs exportados
├─
├─ vpc.tf            # VPC, Subnets, IGW, Route Tables
├─ security-groups.tf # SGs para EC2 e RDS
├─ ec2.tf            # EC2 Instance + User Data
├─ rds.tf            # DB Subnet Group + RDS Instance
├─ s3.tf             # S3 Bucket + Versioning + Encryption
├─ s3-lifecycle.tf   # Lifecycle Rules
├─ s3-policy.tf      # Bucket Policy
├─ iam.tf            # IAM Role + Policy + Instance Profile
├─
├─ terraform.tfvars  # Valores das variáveis (NÃO commitar secrets!)
├─ secrets.tfvars    # Variáveis sensíveis (no .gitignore)
├─
├─ .gitignore        # *.tfvars, .terraform/, *.tfstate*
└─ README.md         # Documentação do projeto
```


.gitignore recomendado

SEGURANÇA

```
# Terraform
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
crash.log
crash.*.log

# Variáveis sensíveis
secrets.tfvars
*.auto.tfvars

# Override files
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# OS
.DS_Store
Thumbs.db
```